

QEMU

Page Content

- [Scope](#)
- [General](#)
- [Termination](#)
- [Serial console redirection](#)
- [Predefine Memory Contents](#)
 - [Set contents](#)
 - [Dump to File](#)
 - [Shared memory with host](#)
 - [Use a file as memory backend](#)
- [Monitor and QEMU Machine Protocol \(QMP\)](#)
- [Debug and Trace](#)
 - [Documentation](#)
 - [Command line options](#)
 - [Inspect Device Tree used in boot process](#)
- [Networking](#)
- [Interfacing with the host or other QEMU instances](#)
- [Run Ubuntu aarch64 cloud image](#)
- [Platforms](#)
 - [RISC-V](#)
 - [Invocation](#)
 - [Startup internals](#)
 - [Host-Target Interface \(HTIF\)](#)
 - [ARM](#)
 - [i.MX6 Sabre Lite](#)
 - [zynq7000](#)
- [Compiling QEMU](#)
 - [Setup environment](#)
 - [Building](#)
- [Contributing to QEMU](#)
- [Attachments](#)

Scope

- This page is about general QEMU topics
- see [TRENTO-S-M on QEMU](#) for specific TRENTO-S-M topics
- see also [Custom QEMU ARM virt platform](#) for modifying QEMU

General

- <https://www.qemu.org>
- docs:
 - <https://wiki.qemu.org/Documentation>
 - <https://en.wikibooks.org/wiki/QEMU>
 - <https://qemu.weilnetz.de/doc/qemu-doc.html>
- Repos:
 - <https://github.com/qemu/qemu>
 - <https://git.qemu.org>
- Mirrored repos
 - <https://bitbucket.hensoldt-cyber.systems/projects/EXT/repos>

Termination

In Linux, processes can be killed with "Ctrl-c", but depending on how stdio is connected to the guest running inside QEMU, it may not work as expected

- if "-serial stdio" is used, the virtual serial port is redirected onto stdio and "Ctrl-c" immediately terminates QEMU
- if "-serial mon:stdio" is used, the virtual serial port and the QEMU monitor are multiplexed onto stdio. The "Ctrl-c" signal will not kill QEMU, but is passed to the guest. QEMU must be terminated via the monitor with "Ctrl-a x", where "Ctrl-a" enters the monitor and "x" is the monitor command for termination.

Serial console redirection

See <https://qemu.weilnetz.de/doc/qemu-doc.html> about what can be specified in "-serial <dev>"

<dev>	description
vc[:WxH]	virtual console. Optionally, a width and height can be given in pixel with c:800x600 It is also possible to specify width or height in characters: vc:80Cx24C
pty	[Linux only] Pseudo TTY (a new PTY is automatically allocated) as /dev/pts/X
none	No device is allocated.
null	void device
/dev/xxx[, baud=y]	[Linux only] Use host tty, e.g. /dev/ttyS0. The host serial port parameters may be set according to the emulated ones or command line settings are used (default baudrate is 115200)
file: <filename>	Write output to file, no character can be read.
pipe: <filename>	name pipe
udp: [remote_host] :remote_port [@[src_ip]: src_port]	• UDP Net Console. When remote_host or src_ip are not specified they default to 0.0.0.0. When not using a specified src_port a random port is automatically chosen. • If you just want a simple readonly console you can use netcat or nc, by starting QEMU with: -serial udp::4555 and nc as: nc -u -l -p 4555. Any time QEMU writes something to that port it will appear in the netconsole session. • If you plan to send characters back via netconsole or you want to stop and start QEMU a lot of times, you should have QEMU use the same source port each time by using something like -serial udp::4555@:4556 to QEMU. Another approach is to use a patched version of netcat which can listen to a TCP port and send and receive characters via udp. If you have a patched version of netcat which activates telnet remote echo and single char transfer, then you can use the following options to set up a netcat redirector to allow telnet on port 5555 to access the QEMU port. • this QEMU options: -serial udp::4555@:4556 • can be accessed via: ◦ netcat -u -P 4555 -L 0.0.0.0:4556 -t -p 5555 -I -T ◦ telnet localhost 5555
tcp:[host]: port[, server] [, nowait][, nodelay][, reconnect=sec onds]	• use a raw TCP socket connection for serial I/O • If host is omitted, 0.0.0.0 is assumed. • there are two modes of operation. ◦ by default, QEMU sends the serial I/O to a host ▪ if "reconnect" option is used, QEMU will attempt to reconnect after the given interval. ◦ "server" make QEMU wait for a connection from client. ▪ QEMU will block waiting for a connection unless the "nowait" option was specified. ▪ ⚠ QEMU supports one connection only. More than one connection is accepted, but only one connection is served, data on all other connection is dropped. • The nodelay option disables the Nagle buffering algorithm. • Examples ◦ send tcp console to 192.168.0.2 port 4444: -serial tcp:192.168.0.2:4444 ◦ listen and wait on port 4444 for connection: -serial tcp::4444, server ◦ don't not wait and listen on ip 192.168.0.100 port 4444: -serial tcp:192.168.0.100:4444, server, nowait
telnet:host: port[, server] [, nowait][, nodelay]	The telnet protocol is used instead of raw tcp sockets. The options work the same as if you had specified -serial tcp. The difference is that the port acts like a telnet server or client using telnet option negotiation. This will also allow you to send the MAGIC_SYSRQ sequence if you use a telnet that supports sending the break sequence. Typically in unix telnet you do it with Control-] and then type "send break" followed by pressing the enter key.
mon: <dev_string>	This is a special option to allow the monitor to be multiplexed onto another serial port. The monitor is accessed with key sequence of "Ctrl+A" and then pressing "C". <dev_string> should be any one of the serial devices specified above. An example to multiplex the monitor onto a telnet server listening on port 4444 would be: -serial mon:telnet::4444, server, nowait When the monitor is multiplexed to stdio in this way, Ctrl+C will not terminate QEMU any more but will be passed to the guest instead.

Predefine Memory Contents

Set contents

This may work in newer version of QEMU only. See <https://github.com/qemu/qemu/blob/master/docs/system/generic-loader.rst>

Parameter	Description
-device loader, addr=<ADDRESS>, data=<DATA>, data-len=<LEN>	set Memory at <ADDRESS> to <DATA>, where length of data is <LEN>
-device loader, file=<FILENAME>, addr=<ADDRESS>	put content of <FILENAME> at <ADDRESS>

Dump to File

Memory can be dumped to a file using the QEMU monitor command

```
pmemsave <ADDRESS> <LEN> <FILENAME>
```

Shared memory with host

https://qemu.weilnetz.de/doc/qemu-doc.html#Inter_002dVM-Shared-Memory-device

Use a file as memory backend

<https://docs.pmem.io/getting-started-guide/creating-development-environments/virtualization/qemu>

Monitor and QEMU Machine Protocol (QMP)

- https://qemu.weilnetz.de/doc/qemu-doc.html#pcsys_005fmonitor
- <https://wiki.qemu.org/Documentation/QMP>
- Parameters are similar to these for [serial console redirection](#)

<code>-monitor <dev></code>	Redirect the monitor to host device dev (same devices as the serial port). The default device is vc in graphical mode and stdio in non graphical mode. Use " <code>-monitor none</code> " to disable the default monitor.
<code>-qmp <dev></code>	Like " <code>-monitor</code> " but opens in 'control' mode.
<code>-qmp-pretty <dev></code>	Like " <code>-qmp</code> " but uses pretty JSON formatting.
<code>-mon [chardev=]name[, mode=readline control][, pretty=[on off]]</code>	Setup monitor on chardev name. pretty turns on JSON pretty printing easing human reading and debugging.
<code>-monitor telnet:127.0.0.1:55555,server, nowait</code>	Start Qemu with a telnet server, that enables gdb-like debugging. A list of commands can be found here (these commands are for Qemu to 4.1.0; you might require to update your version).

Debug and Trace

Documentation

- <https://en.wikibooks.org/wiki/QEMU/Invocation>
- <https://stackoverflow.com/questions/19778785/how-to-use-single-step-mode-in-qemu>
- <http://kaneton.opaak.org/wiki/debug.html>

Command line options

<code>-s</code>	shortcut option (for <code>-gdb tcp::1234</code>) to make qemu's gdbserver wait for a connection on port 1234 of the local machine.
<code>-S</code>	freeze the CPU at startup so that it can be started from the debugger (or by typing c on qemu's console).
<code>-singlestep</code>	require for single step debugging, disables all optimization and code translation features and run all machine code asm instruction one-by-one
<code>-D [filename]</code>	write log to file instead of stdout

-d opt1, opt2...	tracing options. Most common way is using "-d in_asm,cpu -singlestep -D qemu_log.txt"	
	out_asm	show generated host assembly code for each compiled TB
	in_asm	show target assembly code for each compiled TB
	op	show micro ops for each compiled TB
	op_opt	show micro ops after optimization
	op_ind	show micro ops before indirect lowering
	int	show interrupts/exceptions in short format
	exec	show trace before each executed TB (lots of logs)
	cpu	show CPU registers before entering a TB (lots of logs)
	mmu	log MMU-related activities
	pcall	x86 only: show protected mode far calls/returns/exceptions
	cpu_reset	show CPU state before CPU resets
	unimp	log unimplemented functionality
	guest_errors	log when the guest OS does something invalid (e.g. accessing a non-existent register)
	page	dump pages at beginning of user mode emulation
	nochain	do not chain compiled TBs so that "exec" and "cpu" show complete traces

Inspect Device Tree used in boot process

QEMU passes (for some platforms) a device tree to the code being booted (= code at the reset vector of the cpu). QEMU provides a command line option to save this device tree as a file in the host. Here is an example how to save the device tree being used in the RISC-V invocation of `seL4_test`:

```
qemu-system-riscv64 -machine spike_v1.10,dumpdtb=virt.out -nographic -s -serial mon:stdio -m size=4095M -
kernel images/sel4test-driver-image-riscv-spike
```

The file `virt.out` can be converted to a text file with :

```
fdtdump virt.out.
```

Networking

- <https://en.wikibooks.org/wiki/QEMU/Networking>
- <https://people.gnome.org/~markmc/qemu-networking.html>

Interfacing with the host or other QEMU instances

- `ivshmem`:
 - http://people.redhat.com/pbonzini/qemu-test-doc/_build/html/topics/pcsys_005fother_005fdevs.html
 - <https://github.com/qemu/qemu/blob/master/docs/specs/ivshmem-spec.txt>
 - `ivshmem` seems not straight forward to be used for host guest communication because of the information and experiences described in <http://iopscience.iop.org/article/10.1088/1742-6596/960/1/012015>
 - ToDo: how is this related to VirtIO (<https://wiki.osdev.org/Virtio>)?
- serial UART
 - <https://serverfault.com/questions/760668/serial-communication-between-qemu-host-and-guest>
 - in the [HAR Demo](#), the 2nd serial port is connected to either a pseudo-terminal or a TCP socket on the host.
- other
 - <https://github.com/texane/vpcie>
 - <https://stackoverflow.com/questions/44254790/how-to-create-out-of-tree-qemu-devices>

Run Ubuntu aarch64 cloud image

- instructions taken from
 - <https://www.cnx-software.com/2016/05/10/how-to-run-ubuntu-16-04-aarch64-64-bit-arm-cloud-images-on-your-intelamd-linux-computer>
 - <https://gist.github.com/oznu/ac9efae7c24fd1f37f1d933254587aa4>
 - <https://gist.github.com/george-hawkins/16ee37063213f348a17717a7007d2c79>
- Steps to create the image:
 - OS image

```
wget https://cloud-images.ubuntu.com/releases/18.10/release/ubuntu-18.10-server-cloudimg-arm64.img
```

- create an overlay image to be used, so the original image will not be modified

```
qemu-img create -b ubuntu-18.10-server-cloudimg-arm64.img -f qcow2 ubuntu-18.10-server-cloudimg-arm64.img.ovl
```

- EFI

- Get QEMU EFI

```
wget https://releases.linaro.org/components/kernel/uefi-linaro/latest/release/qemu64/QEMU_EFI.fd
```

- Create EFI volumes:

```
dd if=/dev/zero of=flash0.img bs=1M count=64
dd if=QEMU_EFI.fd of=flash0.img conv=notrunc
dd if=/dev/zero of=flash1.img bs=1M count=64
```

- cloud config

- create a SSH key The given SSH key referenced below is a ED25519 key without a password ([ssh_id_ed25519_testuser.prv](#), [ssh_id_ed25519_testuser.pub](#)) created using

```
ssh-keygen -o -a 100 -t ed25519
```

- define a password for manual login. The password below is "secret" and the has is created using

```
openssl passwd -1 -salt SaltSalt
```

or

```
mkpasswd --method=MD5 --salt=SaltSalt
```

- create a cloud.cfg file. See <https://cloudinit.readthedocs.io/en/latest/topics/examples.html> for format details.

```
#cloud-config
users:
  - name: testuser
    lock-passwd: false
    passwd: $1$SaltSalt$YhgRYajLPrYevs14poKBQ0
    ssh-authorized-keys:
      - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMP6k00eXgF5mat0Zl2DeTsf5+xpw+aKePI8kBkXcswV
    sudo: ['ALL=(ALL) NOPASSWD:ALL']
    groups: sudo
    shell: /bin/bash
```

- Create a cloud-config disk image:

```
cloud-localds --disk-format qcow2 cloud.img cloud.cfg
```

- Launch QEMU:

```
qemu-system-aarch64 -m 1024 -smp 2 -cpu cortex-a57 -M virt -nographic \
-drive if=pflash,format=raw,file=flash0.img \
-drive if=pflash,format=raw,file=flash1.img \
-drive if=none,file=ubuntu-18.10-server-cloudimg-arm64.img.ovl,id=hd0 \
-device virtio-blk-device,drive=hd0 \
-drive if=none,id=cloud,file=cloud.img \
-device virtio-blk-device,drive=cloud \
-netdev user,id=user0,hostfwd=tcp::2222-:22 \
-device virtio-net-device,netdev=user0
```

- Things to do

```
# for some reason, mc is missing in the packet source from http://ports.ubuntu.com/ubuntu-ports
echo "deb http://ftp.hosteurope.de/mirror/archive.ubuntu.com bionic main restricted universe multiverse"
> /etc/apt/sources.list.d/official.list
apt-get update
apt-get install mc

# changes for verbose boot output:
# /etc/default/grub: change GRUB_CMDLINE_LINUX_DEFAULT="quiet splash" to ""
# /etc/grub.d/10_linux: change quiet_boot="1" to "0"

# root partition size and additional disks
# qemu-img info ubuntu-18.04_bionic-server-cloudimg-arm64.img
# https://superuser.com/questions/1022019/how-to-increase-size-of-an-ubuntu-cloud-image
# https://docs.ovh.com/gb/en/public-cloud/create_and_configure_an_additional_disk_on_an_instance
# https://github.com/number5/cloud-init/blob/master/doc/examples/cloud-config-disk-setup.txt
# https://askubuntu.com/questions/1019772/how-to-setup-partitions-on-my-boot-disk-with-cloud-config
# https://cloudinit.readthedocs.io/en/latest/topics/examples.html#disk-setup
```

Platforms

RISC-V

Invocation

```
qemu-system-riscv64 -machine spike -cpu rv64 -nographic ...
```

Since QEMU 5.x, the system will only start in M-Mode if "-bios <some.elf>" is used. If "-kernel <some.elf>" is used, QEMU tries to load **opensbi-riscv64-generic-fw_dynamic.elf** as "bios" in M-Mode. This file usually comes with QEMU and contains OpenSBI. The name is hard-coded, see <https://github.com/qemu/qemu/blob/master/hw/riscv/spike.c#L255>. The ELF passed as "kernel" is started in S-Mode then and can use the SBI. There is a safeguard in place that the kernel ELF does not try to use the region the M-Mode firmware uses, QEMU will fail to start and the output is something like

```
$ qemu-system-riscv64 -machine spike -cpu rv64 -m size=1024M -nographic -singlestep -kernel /some
/kernel_plus_opensbi.elf
rom: requested regions overlap (rom phdr #0: /some/kernel_plus_opensbi.elf. free=0x0000000080014500,
addr=0x0000000080000000)
qemu-system-riscv64: rom check and register reset failed
```

Startup internals

QEMU uses a fixed internal bootloader at 0x1000, which will start user image at 0x80000000:

```

0x00001000: auipc t0,0           # 0x1000
0x00001004: addi a1,t0,32             #
0x00001008: csrrs a0,mhartid,zero      #
0x0000100c: ld t0,24(t0)              #
0x00001010: jr t0                     # jump to 0x80000000
0080000000: .... user image here

```

Host-Target Interface (HTIF)

The spike platform does not implement a serial console, but there is HTIF instead. This is a set of virtual I/O register implemented via a customizable memory location. QEMU traps accesses to the registers, the driver in the emulate machine must poll them to detect status update. The memory location is defined via a special section on the ELF file passed via "**-bios**" or "**-kernel**", thus HTIF will not work with raw binaries.

ARM

i.MX6 Sabre Lite

- for real hardware see [Sabre Lite \(BD-SL-i.MX6\)](#)
- **CCM_ANALOG_PLL_ENETn**: disabled and powered down on startup, **BYPASS_CLK_SRC=1** (24 MHz oscillator), **DIV_SELECT=1** (50 MHz clock)
- emulated PHY is at address **0x00** with **ID1=0x0007**, **ID2=0xc0d1**, see QEMU source at **hw/net/imx_fec.c**.

zynq7000

tbd ...

Compiling QEMU

Setup environment

```

sudo apt-get install build-essential bison flex python pkg-config libglib-perl zlib1g-dev libglib2.0-dev
libpixman-1-dev

```

Building

```

QEMU_VERSION=7.0.0

QEMU_CONFIG=(
    # restrict which QEMUS are build
    --target-list=aarch64-softmmu,riscv64-softmmu
    # don't require shared library libgtk-3.so.0
    --disable-gtk
    # don't require shared library libpulse.so.0
    --audio-driv-list=""
    # avoid more dependencies
    --disable-brlapi
    --disable-libiscsi
    --disable-libnfs
    --disable-pa
    --disable-rbd
    --disable-sdl
    --disable-snappy
    --disable-vnc
    --disable-vnc-jpeg
    --disable-vnc-png
    --disable-vnc-sasl
)

QEMU_BASE_DIR=qemu-${QEMU_VERSION}
QEMU_TAG=v${QEMU_VERSION}
QEMU_ARCHIVE=qemu-${QEMU_VERSION}.tar.xz

# setup a workspace
mkdir ${QEMU_BASE_DIR}
cd ${QEMU_BASE_DIR}

# download and extract sources
wget https://download.qemu.org/${QEMU_ARCHIVE}
mkdir src
tar -xvf ${QEMU_ARCHIVE} -C src
QEMU_SRC=$(cd src/qemu-${QEMU_VERSION}; pwd)

# build
mkdir build
cd build
${QEMU_SRC}/configure ${QEMU_CONFIG[@]}
make -j 8

```

Contributing to QEMU

- Patches have to be sent to the mailing list, they do not accept pull requests
- <https://sr.ht/> provides an option to create the mail

Attachments

File	Modified
File ssh_id_ed25519_testuser.pub	Nov 06, 2018 by HG123912
File ssh_id_ed25519_testuser.prv	Nov 06, 2018 by HG123912

Drag and drop to upload or [browse for files](#)
[Download All](#)